Scalable Anomaly Detection in Urban Transportation Data Using Apache Spark

Gülçin Hacıosmanoğlu Mehmet Emir Albayrak
Department of Computer Engineering
Gebze Technical University
Turkey

***Abstract*—Urban public transportation systems generate massive volumes of data on a daily basis. These data streams contain valuable information about passenger mobility patterns, system efficiency, and operational anomalies. Detecting unusual patterns in such data is critical for improving service quality, optimizing resource allocation, and ensuring system reliability. In this project, we address the problem of anomaly detection in large-scale public transportation data using a data-driven approach. The dataset is first preprocessed and aggregated to capture meaningful temporal patterns. Then, anomaly detection is performed using statistical techniques based on Z-score normalization. To handle the scale of the data efficiently, the entire pipeline is implemented using Apache Spark in a cloud environment. Additionally, as an extension to the baseline statistical approach, we propose the use of Isolation Forest, a machine learning-based anomaly detection method, to improve detection performance and provide a comparative analysis.**

***Index Terms*—Anomaly detection, Apache Spark, urban transportation, Z-score, Isolation Forest, distributed computing.**

I. Introduction

Urban public transportation systems generate massive volumes of data on a daily basis. These data streams contain valuable information about passenger mobility patterns, system efficiency, and operational anomalies. Detecting unusual patterns in such data is critical for improving service quality, optimizing resource allocation, and ensuring system reliability.

However, analyzing transportation data at scale presents several challenges. Passenger counts vary significantly depending on factors such as time of day, day of the week, and route characteristics. This variability makes it difficult to distinguish between normal fluctuations and true anomalies using simple statistical methods.

In this project, we address the problem of anomaly detection in large-scale public transportation data using a data-driven approach. The dataset is first preprocessed and aggregated to capture meaningful temporal patterns. Then, anomaly detection is performed using statistical techniques based on Z-score normalization.

To handle the scale of the data efficiently, the entire pipeline is implemented using Apache Spark in a cloud environment. This enables distributed processing of millions of records and ensures scalability of the solution.

Additionally, as an extension to the baseline statistical approach, we propose the use of Isolation Forest, a machine learning-based anomaly detection method, to improve detection performance and provide a comparative analysis.

The main contributions of this project can be summarized as follows:

- A scalable data processing pipeline for large-scale transportation data
- Statistical anomaly detection using Z-score
- Visualization and analysis of passenger behavior and anomalies
- A proposed extension using machine learning (Isolation Forest) for improved anomaly detection

The implementation of the project, including preprocessing, aggregation, and anomaly detection pipelines, is available at the following GitHub repository: https://github.com/ghaciosm/BDA-urban-transportation-anomaly-detection

II. Revised Problem Definition

The original project proposal aimed to identify anomalous passenger behavior in Istanbul's public transportation system using statistical and machine learning approaches. At the midterm stage, the problem definition is refined as follows: the project seeks to detect unusual hourly passenger demand patterns in the 2020 Istanbul public transportation dataset through a scalable processing pipeline implemented with Apache Spark. The focus is on building a practical end-to-end workflow that includes data ingestion, preprocessing, aggregation, temporal feature extraction, and anomaly detection. At this stage, Z-score has been selected as the baseline anomaly detection method, while Isolation Forest remains within the project scope as a planned extension for the final phase.

In the baseline formulation of this project, an observation is considered anomalous when its passenger count significantly deviates from the expected behavior of comparable temporal groups. More specifically, anomalies are defined using the Z-score criterion with $|Z| > 3$, where Z-scores are computed within grouped contexts formed by transportation line, hour of day, and day of week.

III. Related Work

Anomaly detection has been widely studied across various domains, including finance, cybersecurity, and industrial monitoring. A comprehensive survey by Chandola et al. [1] defines

anomalies as patterns in data that do not conform to expected behavior and provides a taxonomy of anomaly detection techniques. These approaches are generally categorized into statistical, distance-based, density-based, and machine learning methods.

Among statistical approaches, Z-score-based methods are commonly used due to their simplicity, interpretability, and computational efficiency. These methods evaluate observations based on their deviation from the mean, normalized by the standard deviation, making them particularly suitable for large-scale aggregated datasets. In addition to statistical techniques, machine learning-based methods such as Isolation Forest [2] have been widely adopted for anomaly detection in high-dimensional data, as they isolate anomalies through random partitioning.

Time-series anomaly detection is particularly relevant for transportation systems, where passenger demand exhibits strong temporal patterns such as daily and weekly periodicity. Recent studies [3], [4] emphasize that detecting anomalies in time-dependent data requires accounting for contextual and seasonal variations, as well as grouping observations by comparable temporal segments.

In the context of transportation and mobility data, anomaly detection has been used to identify unusual traffic patterns, disruptions, and demand fluctuations [5]. Previous studies show that anomalies in urban transportation systems are often concentrated in specific time intervals or locations, such as peak hours, special events, or unexpected system disruptions. This supports the use of grouped statistical baselines and route-level analysis, as applied in this project.

In addition, large-scale transportation datasets require efficient and scalable processing frameworks. Apache Spark [6] has been widely used for distributed data processing due to its ability to handle large volumes of data in parallel. Unlike previous studies, this work emphasizes scalable distributed processing using Apache Spark for large-scale urban transportation data.

IV. Dataset Description

This project uses the Hourly Public Transportation Dataset published by the Istanbul Metropolitan Municipality (IMM) Open Data Portal [7]. The dataset contains hourly public transportation records and includes variables such as transition date, transition hour, line, line name, town, transport type, transfer type, and passenger count. These attributes provide a detailed view of passenger activity across both temporal and spatial dimensions.

According to the original project proposal, the dataset may reach approximately 10 million records per month, exceeding 100 million rows when one year of data is combined. This scale makes distributed data processing necessary and strongly motivates the use of Apache Spark for preprocessing and large-scale aggregation.

For the current implementation, monthly CSV files for the year 2020 were collected and organized within the project directory as raw inputs. After preprocessing and hourly aggregation on Google Cloud Dataproc, the resulting Z-score result dataset contained 6,039,872 rows and 15 columns.

TABLE I
Sample Records from the Aggregated and Feature-Enriched Dataset

Datetime	Line	Town	Pax	Pass.	Cnt	Hr	Mo	Day	Wknd
2020-01-01 00:00	YENIKAPI-HACIOSMAN	FATIH	1542	1848	89	0	1	4	0
2020-01-01 00:00	KADIKOY-BEYKOZ	BAKIRKOY	27	34	9	0	1	4	0
2020-01-01 01:00	KADIKOY-PENDIK	BAKIRKOY	68	74	10	1	1	4	0
2020-01-01 04:00	KADIKOY-TUZLA	KARTAL	46	47	12	4	1	4	0
2020-01-01 05:00	HALKALI-GEBZE	KUCUKCEKMECE	50	50	19	5	1	4	0
2020-01-01 06:00	AVCILAR-ZINCIRLIKUYU	SISLI	804	885	49	6	1	4	0
2020-01-01 06:00	BESIKTAS-AYAZAGA	SARIYER	28	28	9	6	1	4	0
2020-01-01 06:00	ESENKENT-MECIDIYEKOY	BAKIRKOY	4	4	3	6	1	4	0

Pax=total_passengers, Pass.=total_passages, Cnt=record_count, Wknd=weekend.

TABLE II
Dataset Summary

Property	Value
Data source	IMM Open Data Portal
Period used	2020
Total rows	6,039,872
Total columns	15
Processing framework	Apache Spark
Cloud platform	Google Cloud Dataproc
Anomaly count	69,143
Anomaly ratio	0.011448

Table II summarizes the main characteristics of the dataset and the processing environment used in this study. including the total data size, processing framework, and anomaly statistics.

V. System Architecture and Design

The proposed system is designed as a scalable and distributed data processing pipeline for anomaly detection in large-scale urban transportation data. The architecture follows a modular and sequential workflow that transforms raw data into actionable anomaly insights.

The pipeline begins with data acquisition from the IMM Open Data Portal, where raw monthly transportation records are collected. These data are then ingested into a distributed processing environment using Apache Spark on Google Cloud Dataproc.

Within the Spark cluster, preprocessing operations are performed, including data cleaning, schema alignment, and construction of a unified datetime representation. The processed data is subsequently aggregated at the hourly level based on key attributes such as datetime, transportation line, line name, and district.

Following aggregation, a feature engineering stage is applied to extract relevant temporal features, including hour-of-day, month, day-of-week, and weekend indicators. At the model layer, anomaly detection is conducted using a baseline statistical approach based on Z-score normalization. As an extension, an Isolation Forest model is incorporated into the architecture to enable machine learning-based anomaly detection in the final phase.

Finally, the detected anomalies are analyzed and presented through visualization and reporting components. The entire pipeline is executed in a distributed cloud environment, ensuring scalability, efficiency, and robustness.

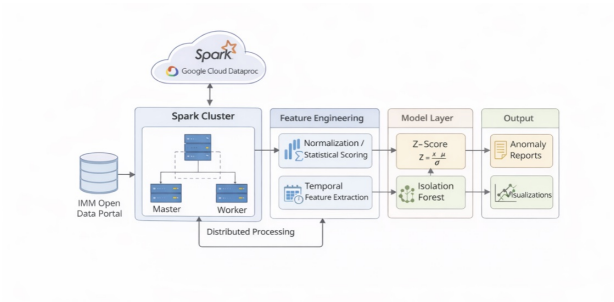

Fig. 1. Scalable anomaly detection architecture based on Apache Spark and Google Cloud Dataproc. The pipeline includes data ingestion, distributed preprocessing, feature engineering, statistical and machine learning-based anomaly detection, and visualization of results.

VI. Implementation Status

At the midterm stage, a significant portion of the proposed system has been successfully implemented. The processing pipeline was implemented using PySpark on a Google Cloud Dataproc cluster consisting of one master node and two worker nodes. Each node was configured with `n1-standard-4` instances, providing a balanced combination of CPU and memory resources for distributed processing.

The implementation proceeded in four stages:

1) Raw monthly transportation data was ingested and preprocessed, including data cleaning, schema alignment, and construction of a unified datetime field.
2) Hourly aggregation was performed using Spark, grouped by datetime, line, line_name, and town.
3) Temporal features such as hour, month, day of week, and weekend indicator were extracted from the datetime field.
4) Anomaly detection was implemented using the Z-score method within grouped contexts (line, hour, day of week).

The final processed dataset contains approximately 6 million rows with anomaly labels. Various visualizations, including hourly trends, anomaly distributions, and line-based comparisons, were generated to interpret the results.

However, the Isolation Forest method has not yet been implemented at the midterm stage. Due to time and computational constraints, this stage is planned for the final phase. The preprocessing and aggregation stages were completed in approximately 10 minutes on the Dataproc cluster. The end-to-end pipeline was completed without execution failures or memory-related issues, demonstrating the practical feasibility and scalability of the proposed approach.

VII. Data Preprocessing and Aggregation

During preprocessing, the `transition_date` and `transition_hour` fields were combined into a single datetime representation. Relevant columns were standardized and numeric variables were converted into appropriate data types. Records with missing or invalid values in key fields were filtered out, along with observations having invalid hour values (outside the 0–23 range) and negative passenger counts.

After preprocessing, the dataset was aggregated at the hourly level using datetime, line, line_name, and town variables. Several temporal features were derived including hour, month, day of week, and weekend indicator.

TABLE III
Main Raw and Derived Variables Used in the Analysis

Raw Variables	Derived Variables
transition_date	datetime
transition_hour	hour
number_of_passenger	total_passengers
number_of_passage	total_passages
line	month
line_name	day of week
town	weekend

VIII. Baseline Anomaly Detection with Z-Score

As the baseline anomaly detection method, the Z-score approach was applied to the aggregated hourly passenger counts. The anomaly score was computed on the `total_passengers` variable. For each observation, the reference group was defined by the combination of line, hour, and day of week, so that each passenger count was evaluated against observations from comparable temporal contexts.

The Z-score is defined as:

$$Z = \frac{x - \mu}{\sigma} \tag{1}$$

where x represents the observed value, μ denotes the mean of the reference group, and σ denotes the standard deviation of the reference group.

Observations with an absolute Z-score greater than 3 were labeled as anomalies. This threshold is commonly used in statistical anomaly detection to identify extreme deviations under the assumption of approximately normal distributions. In the Spark-based implementation, safe division logic was introduced to handle zero-variance groups and avoid divide-by-zero errors.

IX. Results and Discussion

This section presents the results of the Z-score-based anomaly detection and provides an interpretation of the observed patterns. The analysis focuses on temporal trends, dis-

tribution characteristics, and anomaly behavior across different transportation lines.

A. Descriptive Results

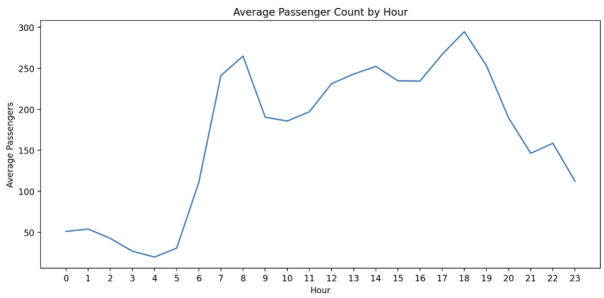

Fig. 2. Average passenger count by hour.

The hourly average passenger count shows a strong diurnal pattern (Fig. 2). Mean passenger counts remain low during early morning hours and rise sharply after 06:00. The highest average levels are observed during commuting periods, with clear peaks in the morning and evening. This confirms that the aggregated dataset preserves systematic temporal demand structure rather than random fluctuation.

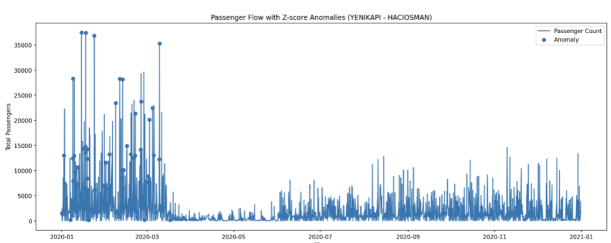

Fig. 3. Passenger flow with detected anomalies for a selected transportation line.

The time-series visualization (Fig. 3) shows substantial variability in passenger counts over time for the selected transportation line. Periods with detected anomalies correspond to observations that deviate strongly from the expected temporal pattern. The concentration of anomalies is higher in some parts of the timeline, suggesting that anomaly behavior may vary across different periods of the year.

Example anomaly. On the YENIKAPI – HACIOSMAN line at 2020-01-01 00:00:00, the observed passenger count was 6215, whereas the corresponding group mean was approximately 822.5 with a standard deviation of 1789.3. This resulted in a Z-score of 3.01, and the observation was therefore labeled as an anomaly. This example shows how the Z-score method can capture unusually high passenger demand relative to the expected behavior of comparable temporal groups.

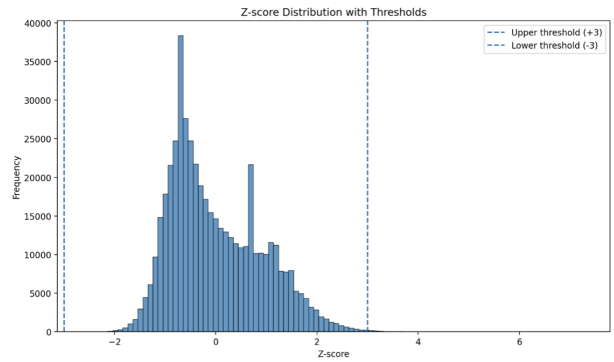

Fig. 4. Z-score distribution with anomaly thresholds.

The threshold-based visualization (Fig. 4) illustrates the decision boundary used for anomaly detection. Observations with Z-score values greater than $+3$ or less than -3 are classified as anomalies. The figure shows that only a small portion of the data falls beyond these thresholds, confirming that anomalies represent rare but significant deviations from normal behavior.

B. Quantitative Evaluation

To complement the visual analysis, a compact quantitative summary was prepared to highlight the main outputs of the baseline anomaly detection pipeline.

TABLE IV
SUMMARY OF PRELIMINARY QUANTITATIVE RESULTS

Metric	Value
Total processed observations	6,039,872
Total detected anomalies	69,143
Overall anomaly ratio	0.011448 (1.14%)
Preprocessing + aggregation time	≈10 minutes
Baseline anomaly threshold	$\lvert Z\rvert > 3$
Anomalies at threshold 2.0	281,601
Anomalies at threshold 2.5	140,063
Anomalies at threshold 3.0	69,143
Month with highest anomaly count	February (24,477)
Line with highest anomaly rate	YENIKAPI–HACIOSMAN (3.57%)

As shown in Table IV, the number of detected anomalies decreases significantly as the Z-score threshold increases, indicating the sensitivity of the method to threshold selection.

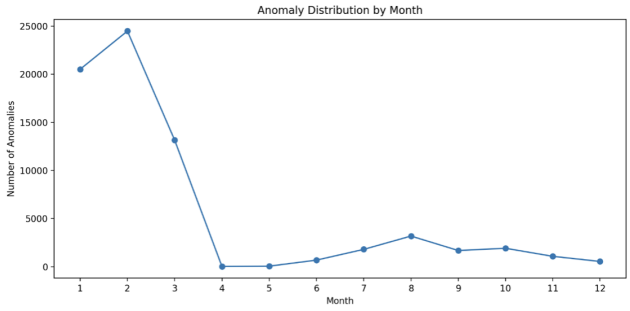

Fig. 5. Monthly distribution of detected anomalies in 2020.

The distribution of anomalies across months (Fig. 5) shows a highly uneven pattern. A significant concentration of anomalies is observed in the first quarter of the year, particularly in January and February. This is followed by a sharp drop in April and May, where anomaly counts approach zero. This pattern may be associated with major changes in urban mobility behavior during 2020, including possible pandemic-related effects. In later months, anomaly counts gradually increase again but remain below early-year levels.

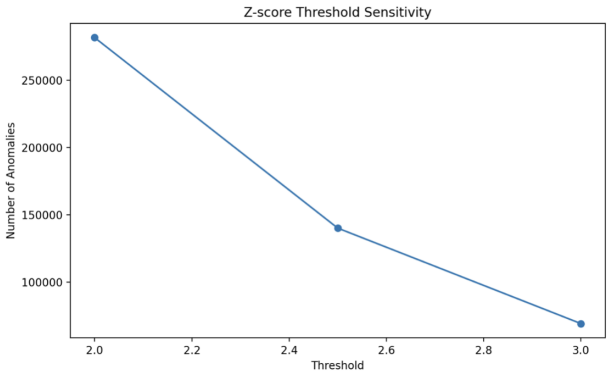

Fig. 6. Sensitivity of anomaly detection to different Z-score thresholds.

The sensitivity analysis (Fig. 6) demonstrates that the number of detected anomalies is highly dependent on the selected Z-score threshold. When the threshold is reduced from 3.0 to 2.0, the number of detected anomalies increases dramatically. This indicates that lower thresholds capture a broader range of deviations, including less extreme fluctuations, while higher thresholds focus only on the most significant outliers.

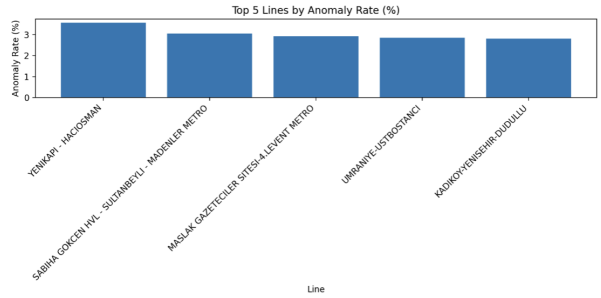

Fig. 7. Top 5 transportation lines with the highest anomaly rates.

The analysis of anomaly rates across transportation lines (Fig. 7) reveals that certain lines consistently exhibit a higher proportion of anomalous observations. Although the differences are not extremely large, the top-ranked lines show noticeably higher anomaly rates than the network average. These lines may correspond to routes with more variable passenger demand, such as transfer-intensive or operationally sensitive connections. This result highlights the importance of line-level analysis, as anomalies are not uniformly distributed across the network.

Overall, the quantitative evaluation shows that the anomaly detection results are sensitive to the selected Z-score threshold, unevenly distributed across months, and concentrated in specific transportation lines. These findings suggest that anomaly behavior in urban transportation data is both temporally and spatially heterogeneous.

Since the dataset does not provide labeled anomaly ground truth, the evaluation is based on descriptive statistics and comparative analysis rather than supervised metrics such as precision and recall.

C. Validation Strategy and Reliability Checks

Since a labeled ground-truth anomaly dataset is not available, the baseline Z-score model was evaluated through a set of reliability-oriented validation checks. First, threshold sensitivity analysis was performed by comparing anomaly counts at $|Z| > 2.0$, 2.5, and 3.0. The results showed that lower thresholds substantially increased the number of flagged observations, whereas $|Z| > 3.0$ produced a more selective anomaly set. Second, the temporal distribution of detected anomalies was analyzed across months. Third, anomaly rates were compared across transportation lines to assess interpretability at the route level. Finally, a manual inspection was conducted for a selected transportation line using time-series visualization. The detected anomaly points corresponded to clear deviations from the typical passenger flow pattern, providing qualitative support for the plausibility of the baseline method.

X. Challenges and Future Work

One of the main challenges in this project is distinguishing between normal temporal variations and true anomalous

behavior. Passenger demand in urban transportation systems naturally fluctuates depending on time of day, day of the week, and location. This variability makes anomaly detection inherently challenging, as extreme values may not always correspond to abnormal events.

Another challenge is the limitation of statistical methods in capturing complex patterns. While the Z-score approach is effective for detecting extreme deviations, it assumes a relatively simple distribution structure and may fail to identify more subtle or context-dependent anomalies.

To address these limitations, future work will focus on integrating the Isolation Forest algorithm as a machine learning-based anomaly detection method. In the final phase, Isolation Forest will be applied on the aggregated dataset using feature-enriched hourly observations. The initial feature set is expected to include passenger count, passage count, record count, hour, month, day of week, and weekend indicator. Due to the large scale of the dataset (over 6 million observations), the method may be applied either on the full dataset or on a reduced representation obtained through filtering, feature selection, or sampling, depending on computational feasibility. The outputs of Isolation Forest will then be compared with the Z-score baseline in terms of anomaly counts, temporal distribution, and interpretability.

XI. Conclusion

This study presented a scalable anomaly detection pipeline for large-scale urban transportation data using Apache Spark. The Z-score method successfully identified unusual passenger demand patterns across temporal and spatial dimensions. The results demonstrate that simple statistical methods, when combined with proper aggregation and grouping strategies, can provide meaningful insights in large-scale data environments. Future work will extend this approach using machine learning-based methods such as Isolation Forest.

References

[1] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," *ACM Computing Surveys*, vol. 41, no. 3, pp. 1–58, 2009.

[2] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proc. IEEE Int. Conf. Data Mining*, 2008, pp. 413–422.

[3] Z. Z. Darban, G. I. Webb, and S. J. Pan, "Deep learning for time series anomaly detection: A survey," *arXiv preprint arXiv:2211.05244*, 2024.

[4] Q. Wang, H. Mao, B. Wang, and L. Guo, "Spatio-temporal anomaly detection in traffic data," in *Proc. ACM CIKM*, 2018.

[5] Y. Hu et al., "Detecting extreme traffic events via a context-augmented approach," *ACM TIST*, 2022.

[6] Apache Software Foundation, "Apache Spark," 2023. [Online]. Available: https://spark.apache.org

[7] Istanbul Metropolitan Municipality, "Hourly public transportation dataset," 2020.